

文献综述：面向文本风格迁移系统的后端服务与数据持久化技术

王浩楠 张博淞 范智杰 何顺康 陈佳铭

2026 年 6 月 19 日

摘要

在文本风格迁移系统的工程化落地中，后端服务承担着业务逻辑编排、数据持久化、用户认证和模型推理调度等核心职责。本文围绕中文多角色文本风格迁移演示平台的系统实现需求，从 Web 服务框架、对象关系映射、用户认证与安全、数据持久化设计以及后端性能优化五个维度梳理相关技术文献。通过对比 FastAPI、Flask 和 Django 在 AI 推理场景下的适用性，分析 SQLAlchemy 的 ORM 设计模式与查询优化策略，总结 JWT+bcrypt 认证方案的工程实践，探讨用户反馈数据在模型持续优化中的价值，并指出现有技术在科研项目快速迭代场景下的不足之处。本文结合本项目五角色并行推理、多模型对比展示和增量训练数据积累的实际需求，阐明了各技术选型的依据和具体实现路径，为系统的后端架构设计提供理论支撑。

目录

1 引言

在文本风格迁移系统的工程化落地中，后端服务扮演着承上启下的关键角色。前端负责用户交互和结果展示，模型服务负责推理计算，而后端则负责将二者串联起来——接收前端请求、调用模型服务、持久化操作记录、管理用户状态。对于本项目的五角色风格迁移演示平台而言，后端系统需要满足以下几项核心需求。

第一，支持并发推理。五类角色的改写请求同时发出时，后端需要能够并行调度，避免串行排队导致用户等待时间过长。第二，提供清晰的 API 规范。前后端分离开发模式下，接口文档的完整性和准确性直接影响协作效率。第三，保障用户数据安全。系统需要支持用户注册登录，并对密码等敏感信息进行加密存储。第四，积累高质量训练数据。每一次用户采纳或修改都是对模型输出的反馈，后端需要系统化地采集这些数据，为后续的增量训练提供语料。

围绕上述需求，本文从 Web 服务框架选型、对象关系映射、用户认证与安全、数据持久化设计以及后端性能优化五个维度展开文献梳理，并在现有技术的基础上分析其在本项目场景中的适用性与局限性。

2 Web 服务框架：FastAPI 的异步特性与 AI 推理场景适配

2.1 主流 Python Web 框架的架构差异

在 Python 生态中，Flask、Django 和 FastAPI 是最具代表性的三个 Web 框架，各自的设计哲学和适用场景存在显著差异。

Flask 被广泛归类为“微框架”，其核心设计理念是保持简洁和灵活。Flask 只提供路由、请求响应和模板渲染等最基础的功能，其他功能（如 ORM、表单验证、用户认证）通过扩展插件按需集成。这种设计使得 Flask 的学习曲线平缓，非常适合小型项目和快速原型开发。然而，Flask 基于 WSGI（Web Server Gateway Interface）规范，采用同步请求处理模型。每个请求会占用一个工作进程或线程，当处理需要长时间运行的任务时，如果并发请求数超过工作进程数，后续请求将被阻塞。

Django 遵循“大而全”的设计哲学，内置了 ORM、模板引擎、表单处理、Admin 后台和用户认证等一站式解决方案。Django 的生态系统极为丰富，适合构建需要复杂业务逻辑的大型应用。但与 Flask 相同，Django 也基于 WSGI 规范，在同步请求处理模型下面临同样的并发瓶颈。

FastAPI 是相对较新的框架，其核心设计基于两个关键选择：一是运行于 ASGI（Asynchronous Server Gateway Interface）协议之上，支持原生异步编程；二是深度集成 Pydantic，利用 Python 类型注解实现自动数据校验和序列化。ASGI 协议允许在单个进程内并发处理多个请求，当某个请求处于 IO 等待状态时，事件循环可以切换到其他

请求继续处理。对于 AI 推理场景，这种异步能力尤其重要——模型推理虽然占用 GPU 计算资源，但请求从接收到模型输出之间存在多个 IO 等待点（如数据预处理、模型加载、结果后处理），异步框架能够更高效地利用这些等待间隙。

2.2 性能基准与工程实践

TechEmpower 的 Web 框架性能基准测试提供了客观的数据支撑。在 JSON 序列化测试中，FastAPI 的每秒请求处理数约为 Flask 的 3 倍。在数据库查询测试中，FastAPI 配合异步数据库驱动（如 `asyncpg`、`databases`）的吞吐量优势更加显著。

除了原始性能数据，FastAPI 在工程实践中的优势同样值得关注。第一，自动生成的 OpenAPI 文档。开发者只需定义 Pydantic 模型，FastAPI 即可生成符合 OpenAPI 规范的交互式文档（Swagger UI 和 ReDoc），前端开发者无需等待后端编写文档即可开始联调。第二，依赖注入系统。FastAPI 的 Depends 机制允许开发者将认证、数据库会话、配置等依赖项声明为可复用的函数，路由函数通过声明依赖即可获得所需对象，代码更加模块化和可测试。第三，Pydantic 的校验能力。Pydantic 模型支持嵌套结构、自定义验证器和类型转换，能够在校验阶段拦截格式错误、缺失字段和非法值，避免这些错误进入业务逻辑层。

2.3 对本项目的启发

本项目后端选用 FastAPI 作为核心框架，主要基于以下考虑。

第一，五角色并行推理的场景要求框架具备良好的并发能力。如果使用 Flask 或 Django，五个角色的改写请求只能串行处理，假设每个请求平均耗时 1.5 秒，总响应时间将达到 7.5 秒，用户体验极差。FastAPI 的异步事件循环允许在等待某个角色推理完成的同时，继续接收和处理其他角色的请求，配合模型服务的全局单例加载策略，能够将总响应时间压缩到接近单个请求的耗时。

第二，前后端分离开发模式下，清晰的 API 文档是高效协作的基础。FastAPI 自动生成的 OpenAPI 文档使前端开发者可以随时查阅接口的请求格式、字段类型和响应结构，减少了反复沟通的成本。

第三，本项目涉及的用户数据较为敏感，需要在接口层进行严格的输入校验。Pydantic 模型能够确保前端传入的角色代码、模型代码和文本长度符合预期，防止非法输入进入后续的业务逻辑和模型推理层。

在实际实现中，本项目将 API 设计为 RESTful 风格，主要包括以下端点：POST `/api/auth/register`（用户注册）、POST `/api/auth/login`（用户登录）、GET `/api/meta/roles`（获取角色列表）、GET `/api/meta/models`（获取模型列表）、POST `/api/transfers`（提交改写请求）、GET `/api/transfers/history`（获取历史记录）、POST `/api/accepted-samples`（采纳样本）、GET `/api/accepted-samples`（获取采纳样本列表，管理员权限）。每个端点都有对应的 Pydantic 模型定义请求体和响应体的结构，并通过 FastAPI 的路由装饰

器关联到具体的处理函数。

3 对象关系映射：SQLAlchemy 的设计模式与查询优化

3.1 ORM 在 Web 应用中的角色

对象关系映射（Object-Relational Mapping, ORM）是连接面向对象编程语言与关系数据库的桥梁。在 Web 应用开发中，ORM 使开发者能够以操作 Python 对象的方式操作数据库记录，将 SQL 语句的生成和执行交由框架自动完成。这种方式带来了两方面的收益：第一，降低了数据库操作的认知负担，开发者无需精通 SQL 语法即可完成常见的增删改查；第二，提高了代码的可移植性，应用可以在不同数据库系统（如 SQLite、PostgreSQL、MySQL）之间切换，只需修改连接配置而无需重写数据访问代码。

然而，ORM 也并非没有代价。自动生成的 SQL 语句在某些复杂场景下可能不够高效，导致查询性能下降；懒加载（lazy loading）机制如果不加控制，可能在循环中触发大量额外查询，造成 N+1 问题。因此，合理使用 ORM 需要开发者理解其背后的 SQL 生成逻辑和加载策略。

3.2 SQLAlchemy 的核心设计

SQLAlchemy 是 Python 生态中最成熟、功能最完备的 ORM 框架，其设计哲学强调“让开发者写出高效的 SQL，而不是隐藏 SQL”。SQLAlchemy 提供了两种核心使用模式。

Core 模式提供了 SQL 构建器和数据库连接池的低层抽象。开发者通过 SQLAlchemy 的表达式语言构建 SQL 语句，该语言与原生 SQL 在结构上接近，但通过 Python 对象封装了表名、字段名和条件，降低了字符串拼接带来的 SQL 注入风险。Core 模式适合需要精细控制 SQL 语句的场景，如复杂联表查询和批量操作。

ORM 模式在 Core 模式之上构建，通过声明式映射（declarative mapping）将 Python 类与数据库表绑定。开发者定义类属性对应表的字段，类的实例对应表中的行。ORM 模式隐藏了 SQL 的细节，使开发者可以通过对象方法（如 `db.add()`、`db.commit()`、`db.execute()`）完成数据操作。

SQLAlchemy 的查询接口设计非常灵活。开发者可以使用 `select()` 函数构建查询，通过链式调用 `where()`、`join()`、`order_by()` 等方法添加条件和排序规则。在关联加载方面，SQLAlchemy 提供了三种策略。懒加载（`lazy='select'`）在访问关联属性时才触发额外查询，适用于不总是需要关联数据的场景。预加载（`lazy='joined'`）通过 LEFT JOIN 一次性获取主表和关联表数据，适用于需要同时展示主表和关联数据的列表页面。子查询预加载（`lazy='subquery'`）通过子查询一次性加载关联数据，适用于分页场景。合理选择加载策略可以将多次查询合并为一次，显著提升接口响应速度。

3.3 迁移管理与版本控制

在科研项目中,数据模型往往随着实验的推进而频繁调整。Alembic 作为 SQLAlchemy 官方推荐的迁移工具,提供了版本化的数据库变更管理。开发者修改模型类后,通过 `alembic revision --autogenerate` 命令自动生成迁移脚本,再通过 `alembic upgrade head` 将变更应用到数据库。

Alembic 的迁移脚本是可编程的 Python 文件,包含 `upgrade()` 和 `downgrade()` 两个方法,分别定义了如何应用变更和如何回滚变更。这种设计使开发者可以在自动生成的基础上手动调整迁移逻辑,例如添加数据迁移(如填充新字段的默认值)、修改外键约束等。迁移脚本被按时间顺序编号并存储在 `versions/` 目录下,使表结构的变更历史可追溯、可回滚。

3.4 对本项目的启发

本项目使用 SQLAlchemy 作为 ORM 框架,采用声明式映射方式定义了五张核心表: `User` (用户信息)、`RoleOption` (角色选项,预置老板、同事、好朋友、女朋友、母亲五类)、`ModelOption` (模型选项,预置 DeepSeek 教师模型、Transformer 规则基线、LoRA 微调模型三类)、`TransferRecord` (改写历史记录)和 `AcceptedSample` (采纳样本池)。五张表之间存在明确的关联关系: `TransferRecord` 通过外键关联到 `User`、`RoleOption` 和 `ModelOption`; `AcceptedSample` 关联到 `TransferRecord` 和 `User`,同时冗余存储了角色和模型的名称字段,便于直接导出为训练数据。

在查询优化方面,历史记录列表接口使用了 `joinedload` 预加载策略。如果不使用预加载,查询 20 条历史记录会触发 1 次主查询、20 次角色查询和 20 次模型查询,总计 41 次查询。通过 `options(joinedload(TransferRecord.role), joinedload(TransferRecord.model))`,所有关联数据在一次查询中完成,大幅减少了数据库往返次数。采纳样本导出功能则利用了 SQLAlchemy 的灵活查询能力,通过筛选 `review_status='approved'` 的记录,直接输出为 JSONL 格式,无需额外编写 SQL 语句。

在迁移管理方面,项目集成了 Alembic 并规范了迁移脚本的生成流程。对于简单变更(如增加字段、修改字段类型),使用 `alembic revision --autogenerate` 自动生成;对于复杂变更(如字段重命名、外键约束调整),在自动生成的基础上手动修改迁移脚本,确保 `upgrade` 和 `downgrade` 方法的正确性和可逆性。通过这种方式,项目在开发过程中多次调整表结构均未出现数据丢失或迁移失败的问题。

4 用户认证与密码安全：JWT 无状态认证与 bcrypt 加密实践

4.1 无状态认证的演进与 JWT 的设计原理

Web 应用的认证机制经历了从有状态到无状态的演进。传统的基于 Session 的认证方式在服务端存储会话信息，客户端通过 Cookie 携带会话 ID。这种模式在单机部署中工作良好，但当系统需要水平扩展时，会话信息的共享和同步成为难题——用户第一次请求被路由到服务器 A，第二次请求可能被路由到服务器 B，如果服务器 B 没有该用户的会话信息，用户会被要求重新登录。

JWT (JSON Web Token) 通过将用户信息加密后存储在客户端，使服务端无需维护会话状态即可完成身份验证。JWT 由三部分组成：头部 (Header) 声明签名算法和令牌类型；载荷 (Payload) 包含用户标识 (sub)、签发时间 (iat) 和过期时间 (exp) 等声明；签名 (Signature) 通过密钥对头部和载荷进行哈希运算生成，用于防止令牌被篡改。

JWT 的工作流程为：用户登录成功后，服务端生成 JWT 并返回给客户端；客户端将 JWT 存储在 localStorage 或 Cookie 中；后续请求在 HTTP 头的 Authorization 字段中携带该令牌 (格式为 Bearer <token>)；服务端验证令牌签名、检查过期时间、解析用户信息后完成认证。由于令牌自包含用户信息，服务端无需查询数据库即可完成认证，减少了数据库访问开销。

JWT 的优势在于无状态和可扩展，但同时也面临令牌失效慢、无法主动撤销等局限。常见的缓解策略包括：设置较短的过期时间 (如 15 分钟到 24 小时)，要求用户定期刷新令牌；使用刷新令牌 (Refresh Token) 机制，在访问令牌过期后通过刷新令牌获取新的访问令牌，避免用户频繁重新登录；在服务端维护令牌黑名单，在用户主动注销或修改密码时将令牌加入黑名单。

4.2 密码存储的最佳实践

密码存储是认证安全的核心环节。明文存储密码在任何情况下都是不可接受的——一旦数据库泄露，所有用户的密码将完全暴露。常见的密码保护方案包括哈希加密和加盐。

哈希加密将原始密码通过单向函数转换为固定长度的哈希值。SHA 系列 (如 SHA-256) 是广泛使用的哈希算法，但因其计算速度快，容易受到暴力破解和彩虹表攻击。加盐 (salt) 是在哈希计算前为每个密码生成一个随机字符串，将密码和盐拼接后再进行哈希，有效抵抗彩虹表攻击。

bcrypt 基于 Blowfish 加密算法设计，内置加盐机制且计算速度可调 (通过工作因子 work factor 控制)。bcrypt 的慢速计算特性使其成为抵御暴力破解的理想选择——即使攻击者获得了哈希值，每秒能尝试的密码数量也极其有限。Argon2 是 2015 年密码哈希竞赛的获胜者，提供了更高的安全性和灵活性，但其在 Python 生态中的普及度不如

bcrypt。

在实际工程中，密码存储通常遵循以下原则：使用现代哈希算法（bcrypt 或 Argon2），不自行设计加密方案；为每个密码生成独立的盐值；设置适当的迭代次数或工作因子（如 bcrypt 的 rounds=12）；禁止存储明文密码或可逆加密的密码。

4.3 FastAPI 中的认证依赖注入

在 FastAPI 中，认证逻辑通常通过依赖注入（Depends）实现。开发者定义一个 `get_current_user` 函数，该函数依赖 `HTTPBearer` 自动从请求头中提取 JWT，然后验证签名、解析用户信息、查询数据库确认用户存在且状态正常，最后将用户对象返回给路由函数。

依赖注入的优势在于认证逻辑的复用性和可测试性。所有需要认证的路由只需在装饰器中添加 `current_user: User = Depends(get_current_user)`，即可在当前请求上下文中获得已验证的用户对象。对于需要管理员权限的接口，可以进一步定义 `require_admin` 函数，该函数依赖 `get_current_user`，在获得用户对象后检查 `is_admin` 字段，无权限则抛出 HTTP 403 异常。

4.4 对本项目的启发

本项目使用 JWT+bcrypt 的组合方案实现用户认证。具体实现包括以下环节。

用户注册时，前端发送用户名、姓名和密码。后端使用 `hash_password()` 函数通过 bcrypt 对密码进行哈希处理，然后存储到数据库。用户登录时，后端根据用户名查询用户记录，使用 `verify_password()` 函数比对明文密码和存储的哈希值，匹配则生成 JWT 令牌并返回。JWT 令牌的有效期设置为 24 小时，令牌中包含用户名和管理员标识。

前端将令牌存储在 `localStorage` 中，并在每次 API 请求的 `Authorization` 头中携带。后端通过 `get_current_user` 依赖解析令牌、验证签名和过期时间，并返回用户对象。对于管理员权限的接口（如获取所有用户、审核采纳样本），在路由依赖链中增加 `require_admin`，确保只有管理员才能访问。

这种设计的优势在于：第一，bcrypt 哈希使得即使数据库泄露也无法还原密码；第二，JWT 的无状态特性使系统无需维护会话存储，便于未来水平扩展；第三，依赖注入将认证逻辑与业务逻辑分离，新增接口时只需声明依赖即可获得当前用户，代码简洁且易于测试。

5 数据持久化设计与反馈闭环机制

5.1 数据库在机器学习演示系统中的角色再定位

在传统的 Web 应用开发中，数据库主要用于存储用户信息、业务数据和操作日志，其角色是“记录的仓库”。然而，在机器学习演示系统中，数据库的角色应当被重新定义——它不仅是信息的存储介质，更是高质量训练数据的“前哨站”。

每一次用户发起的改写请求、每一次用户对改写结果的采纳或编辑，都是对模型输出质量的隐式或显式评价。如果这些交互数据被系统化地采集、存储和审核，就可以转化为模型的增量训练数据，形成“用得越多、效果越好”的正向循环。这一视角将演示系统从静态的模型展示橱窗升级为动态的数据积累平台。

5.2 数据采集的粒度与表结构设计

系统化的数据采集需要明确记录哪些字段。从功能需求出发，每一次改写请求至少应记录以下信息：用户标识（关联登录账户，用于追踪用户行为）、目标角色（用于分析不同角色的表现差异）、所用模型（用于比较不同模型的效果）、输入文本和输出文本（构成训练样本对）、请求时间（用于时间维度分析）以及用户后续操作（如采纳、编辑、重新生成，用于判断输出质量）。

从数据库设计的角度，这些数据应以规范化方式存储。用户信息独立成表，角色选项和模型选项独立成表，改写记录作为核心事实表，通过外键关联上述元数据。这种设计避免了数据冗余，使角色和模型的修改只需在元数据表中更新，所有关联记录自动反映变更。

对于采纳样本，单独建表存储并通过 `transfer_id` 关联回原始改写记录。同时，为了便于直接导出为训练数据，采纳样本表可以冗余存储角色代码、角色名称、模型代码、模型名称、源文本和改写结果。这种冗余设计虽然违反了第三范式，但在实际工程中大幅简化了数据导出流程——导出脚本无需跨表关联即可获得完整的训练样本。

5.3 审核流程与增量训练的数据准备

用户采纳的样本不应直接进入训练集，而应先存入采纳样本池并标记为“待审核”状态（pending），由管理员定期复核。这一审核环节是保证增量训练数据质量的关键。

审核流程通常包括以下步骤：第一，管理员登录系统，查看待审核的采纳样本列表，列表按时间倒序排列，便于优先处理最新数据。第二，管理员逐一检查样本的语义保持（是否保留了原句的核心意思）、角色适配（是否符合目标对象的表达习惯）和自然度（是否存在生硬或模板化问题）。第三，审核通过的样本标记为“已批准”（approved），并进入可导出队列；审核不通过的标记为“已拒绝”（rejected），保留记录但排除在训练集之外。第四，当已批准的样本积累到一定规模（如 100 条以上）时，管理员触发导出

脚本，将样本转换为与训练脚本输入格式一致的 JSONL 文件。第五，导出的文件与现有训练集合并，启动 LoRA 增量训练任务，训练完成后部署更新后的模型。

这一闭环流程使得模型能够利用用户交互中产生的反馈数据不断优化，特别适用于小样本场景下模型的渐进式改进。

5.4 对本项目的启发

本项目在数据库设计中充分体现了反馈闭环的架构思想。`transfer_records` 表记录了每次改写请求的完整信息，包括用户 ID、角色 ID、模型 ID、源文本、改写结果和创建时间，构成了操作审计的完整日志。`accepted_samples` 表独立存储用户采纳的改写结果，通过 `transfer_id` 关联回原始记录，同时冗余存储角色代码、角色名称、模型代码、模型名称、源文本和改写结果，便于直接导出。该表包含 `review_status` 字段，支持 `pending/approved/rejected` 三种状态，管理员可通过界面按状态筛选和审核。

采纳样本导出脚本的设计遵循了数据管道 (data pipeline) 的理念：通过 SQLAlchemy 查询 `review_status='approved'` 的记录，遍历结果集构造训练样本对象 (包含 `role_code`、`source_text`、`rewritten_text` 三个字段)，追加到 JSONL 文件中。导出的文件与现有训练数据合并后，可以直接用于 LoRA 微调脚本的输入，无需额外格式转换。这种端到端的设计使从用户反馈到模型更新的链路尽可能短，降低了数据流过程中的人为错误和格式不兼容风险。

表 1: 采纳样本审核与导出流程

阶段	操作	技术实现
采集	用户点击“采纳”按钮	前端调用 POST <code>/api/accepted-samples</code> ，后端写入 <code>accepted_samples</code> 表
审核	管理员查看待审核样本	前端展示 <code>review_status=pending</code> 列表，管理员逐一审批
导出	管理员触发导出操作	脚本查询 <code>approved</code> 记录，输出为 JSONL 文件
训练	增量微调	新数据与旧训练集合并，启动 LoRA 训练
部署	模型更新	训练完成后替换模型权重文件，重启推理服务

6 现有研究的不足与本项目的改进方向

尽管上述技术在各自领域已较为成熟，但在面向中文文本风格迁移的 Web 系统实现中，仍存在若干未被充分解决的问题。

第一，关于 FastAPI 在 AI 推理场景下的性能调优，现有文献多集中于框架本身的基准测试，而缺少针对“多角色并行推理”这一特殊场景的工程实践指导。例如，当多个推理请求并发时，Uvicorn 的工作进程数如何配置、异步任务的超时时间如何设定、请求级别的模型缓存策略如何实现，这些问题在官方文档中虽有提及，但缺乏系统的案例参考和最佳实践总结。此外，FastAPI 的异步机制虽然能够处理并发请求，但如果模型推理本身是同步阻塞操作（如使用非异步的 Transformers 库），异步的优势将被抵消，需要配合线程池或进程池来真正实现并行。

第二，关于 SQLAlchemy 在科研项目中的迁移管理，现有资料多面向长期维护的企业级应用，对于数据模型频繁调整的科研场景，迁移脚本的生成策略和冲突解决机制讨论不足。在快速迭代的科研环境中，表结构的变更可能涉及字段重命名、类型变更和外键约束调整，Alembic 的自动生成功能在此类场景下时常产生不准确的迁移代码，需要开发者手动修正。此外，当多个开发者并行修改模型时，迁移脚本的合并和冲突解决也缺乏成熟的指导方案。

第三，关于用户反馈数据的闭环利用，已有不少研究提出了“数据飞轮”的概念，但在小型科研项目中的落地路径尚不清晰。大部分系统止步于数据采集，缺乏从采集到审核到导出的完整工具链。数据格式的标准化、审核界面的便捷性、导出流程的自动化，这些环节的缺失导致数据闭环的构想难以真正实现。此外，用户反馈数据的质量控制也是一个被忽视的问题——并非所有用户采纳的样本都是高质量的，如何设计有效的审核机制来过滤低质量样本，缺乏系统的讨论。

第四，关于数据库连接池的管理和优化，在并发请求场景下，数据库连接数可能成为系统瓶颈。SQLAlchemy 内置的连接池机制虽然提供了基本的连接复用，但在高并发场景下如何合理配置连接池大小、如何处理连接超时和泄露问题，实践案例较少。

对本项目的改进方向：本项目在上述四个方向进行了针对性探索。在并发优化方面，后端通过独立的模型服务模块（`model_runner.py`）封装了模型加载和推理逻辑，使用全局单例模式避免每个请求重复加载模型权重；对于同步的 Transformers 推理，通过 `run_in_executor` 将其委托给线程池执行，确保主事件循环不被阻塞。在迁移管理方面，项目规范了迁移脚本的生成流程，对于复杂变更采用手动编写迁移脚本而非完全依赖 `--autogenerate`，并通过 `upgrade` 和 `downgrade` 方法保证迁移的可逆性；在团队协作中，要求开发者在修改模型后先拉取最新迁移脚本再生成新迁移，减少合并冲突。在数据闭环方面，本项目不仅实现了采纳样本的存储和审核，还提供了标准化的导出脚本（`export_samples.py`），将已批准的样本自动转换为与训练脚本输入格式一致的 JSONL 文件；审核界面提供了按状态筛选、批量操作和快速预览功能，提升了审核效率。在数据库连接池方面，本项目配置了适当的连接池大小（`pool_size=10, max_overflow=20`），

并设置了连接回收时间，避免长时间闲置连接被数据库服务端关闭。

表 2: 现有研究不足与本项目改进方向

研究方向	现有研究不足	本项目改进策略
FastAPI 并发调优	缺乏多角色推理场景的工程指导	模型单例加载 + 线程池隔离同步推理
SQLAlchemy 迁移管理	科研项目快速迭代场景支持不足	规范迁移流程 + 复杂变更手动编写
数据闭环落地	从采集到导出的完整工具链缺失	标准化导出脚本 + JSONL 无缝对接训练
连接池管理	高并发场景配置案例较少	合理配置 pool_size + 连接回收策略

7 小结

本节从 Web 服务框架、对象关系映射、用户认证与安全、数据持久化设计以及性能优化五个维度，系统梳理了本项目后端开发所涉及的核心技术。FastAPI 的异步特性为本项目多角色并行推理提供了性能保障，其自动文档生成和 Pydantic 校验机制提升了前后端协作效率和接口安全性。SQLAlchemy 的 ORM 能力和迁移管理为数据模型的灵活调整提供了支撑，预加载策略有效解决了 N+1 查询问题。JWT+bcrypt 的组合方案实现了安全可靠的无状态认证，依赖注入模式使认证逻辑与业务逻辑清晰分离。基于采纳样本的审核和导出机制构建了从用户反馈到模型迭代的完整数据闭环，使演示系统兼具展示和数据积累的双重功能。

上述技术的综合运用，使本项目后端能够在保障服务稳定性的同时，为模型的持续优化积累高质量的训练语料，为后续的研究扩展和工程迭代奠定基础。

参考文献

[1] FastAPI Contributors, “FastAPI Documentation,” 2024. <https://fastapi.tiangolo.com/>

[2] SQLAlchemy Authors, “SQLAlchemy ORM Documentation,” 2024. <https://docs.sqlalchemy.org/>

[3] Alembic Contributors, “Alembic: Database Migrations,” 2024. <https://alembic.sqlalchemy.org/>

- [4] JWT.io, “JSON Web Token Introduction,” 2023. <https://jwt.io/>
- [5] bcrypt Developers, “bcrypt: Password Hashing for Python,” 2023. <https://github.com/pyca/bcrypt/>
- [6] 中国信息通信研究院, “人工智能研发运营体系（MLOps）实践指南（2023 年）,” 2023.
- [7] TechEmpower, “Web Framework Benchmarks Round 21,” 2024. <https://www.techempower.com/benchmarks/>
- [8] R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” PhD Thesis, UC Irvine, 2000.